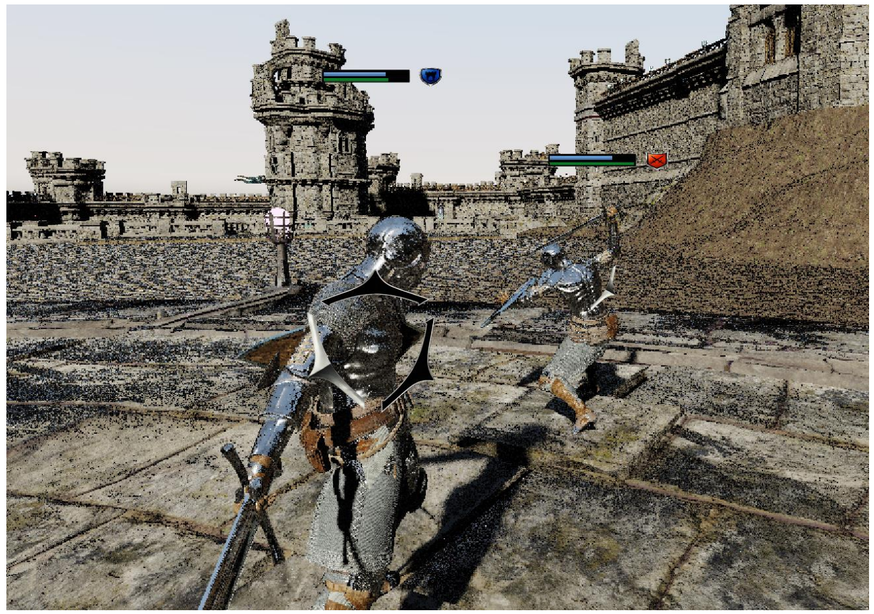


Game Client Programming Portfolio

# 클라이언트 프로그래밍 포트폴리오


**이다건**

010 – 5954 – 0403

leedageon764@gmail.com

# 목차

## ❖ 이력

## ❖ EisenValor 졸업작품

- Inverse Kinematics & PIX profiling
- Client-Server 공격-피격 타이밍 동기화
- Combat Input Architecture
- Finite State Machine

## ❖ Helicopter Simulation 3DGameProgramming Project

- Motion Blur
- Tessellation

## ❖ 기타 프로젝트

- **TinyMMORPG**
  IOCP Game Server
- **Zombie Shooting Game**
  Unreal Engine Blueprint
- **NightMareAdventure**
  Python

# 이력



이다건

C++20 기반
게임 클라이언트 프로그래머

## 프로필

생년월일 2000.04.03
주소 서울시 구로구
연락처 010-5954-0403
이메일 leedageon764@gmail.com

## 학력

2023.03 ~ 2027.02
한국공학대학교 게임공학과

## LANGUAGE

C++20

## CLIENT

Win32API
OpenGL
DirectX12

## SERVER

IOCP

## USED AI TOOLS

CURSOR API
GEMINI CLI
CODEX CLI

## ASSISTANT

2024.03 ~ 2026.12
선형대수학, 이산수학 수업 조교

2026.03 ~ 06
윈도우프로그래밍 수업 조교

2026.09 ~ 12
컴퓨터그래픽스 수업 조교

## PROJECTS

2025.10 ~ 2025.12
Helicopter Simulation

2025.12 ~ 2026.06
EisenValor

2026.05 ~ 2026.06
TinyMMORPG

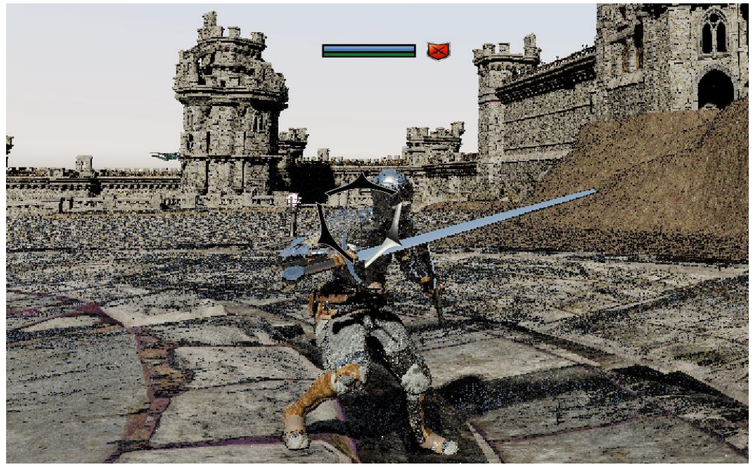
## ACADEMIC ACTIVITIES

2026.05
한국게임학회 논문집 수록 및 포스터 발표

https://github.com/Lee-Dageon

# EisenValor





| | |
|---|---|
| 프로젝트 이름 | EisenValor |
| 장르 | 3D 중세 전략 액션 PVP |
| 설명 | For Honor식 3방향 스탠스 전투와 점령지를 결합한<br>3 vs 3 실시간 멀티플레이 대전 액션 게임 |
| 개발 인원 | 3명<br>프레임워크/렌더링, 서버, 클라이언트/애니메이션 |
| 담당 역할 | 클라이언트/애니메이션<br>UI, 클라이언트 FSM, IK 기반 발 보정, 서버 패킷 반영<br>모델 및 애니메이션 편집&화면 렌더링&연동, 게임사운드 연동 |
| 개발 언어 | C++20, HLSL |
| 사용 툴 | VS 2022, C++20, DirectX12, Unity, Unreal, Git, Cascadeur<br>Cursor, Gemini CLI, Codex CLI |
| 제작 기간 | 2025.12 ~ 2026.06 |

# Inverse Kinematics

## 개요

### 무엇을 구현했나

전투용 IDLE, 이동, 회피 상태 위에 Foot IK를 적용해 발이 지형에 맞게 자연스럽게 보이도록 한다. 무릎 방향, pelvis 높이, 애니메이션 보간까지 함께 다룬다.

### 왜 필요한가

원본 애니메이션만 사용하면 계단, 경사면, 낮은 지형에서 발이 뜨거나 무릎이 비정상적으로 꺾인다. 따라서 런타임 지면 정보를 받아 자세를 보정한다.

- AnimationComponent가 먼저 원본 본 행렬을 계산하고, 이를 pre-IK global matrix로 별도 보존한다.
- 이후 FootIKComponent가 각 발 본의 world 위치를 기준으로 아래 방향 ray를 쏴 지면 hit를 탐색하고,

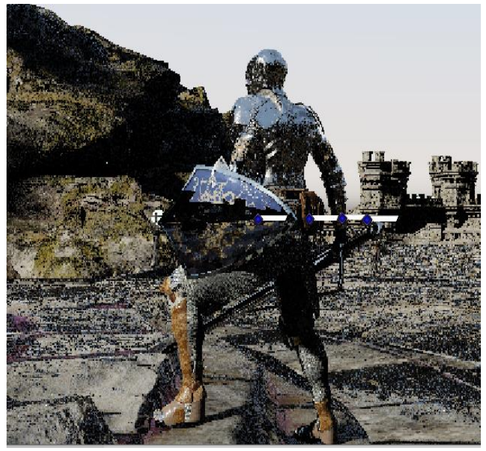
그 결과를 이용해 발 목표 위치와 pelvis offset을 계산한다.

- 이렇게 만든 목표값은 IKTarget에 thigh / calf / foot 체인 정보와 함께 정리되고,

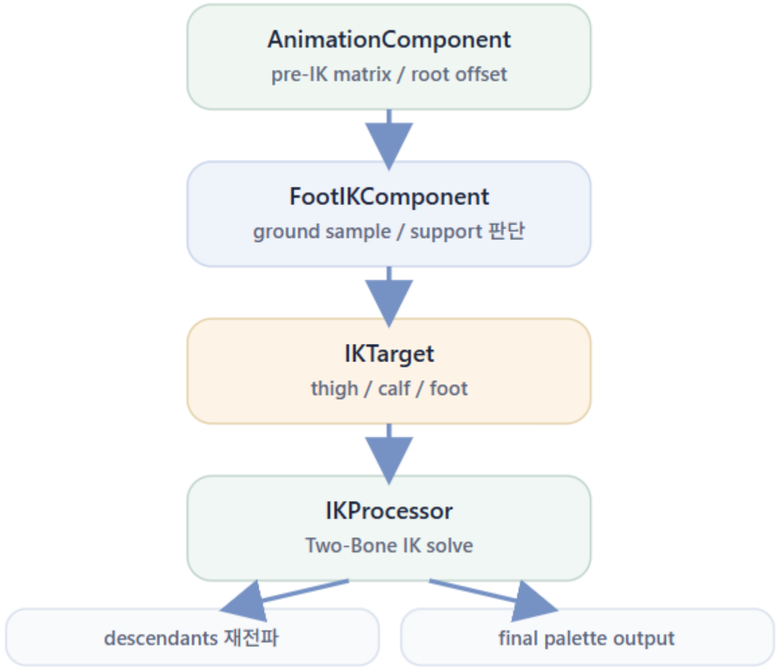
IKProcessor가 Two-Bone IK를 이용해 허벅지-종아리-발 구조를 보정한다.

- IK 적용이 끝나면 수정된 본의 자식 계층을 다시 전파하고, Final Palette를 갱신해 스키닝에 사용할 최종 본 팔레트를 만든다.









# Inverse Kinematics

## ❖ 문제와 해결



지면 샘플 결과를 즉시 반영하면서 발 위치와 가중치가 프레임마다 급하게 변해
계단과 경사면에서 순간적으로 무릎이 반대로 꺾이는 문제가 발생했다.



원본 애니메이션의 pre-IK 본 행렬을 별도로 저장하고, 허벅지-종아리 방향을 pole vector 힌트로
사용해 무릎의 굽힘 방향을 안정화했다.
SmoothApproach를 적용해 발 weight와 pelvis offset이 목표 값으로 한 번에 점프하지 않고
시간에 따라 점진적으로 수렴하도록 만들어 IK 결과의 시각적 흔들림을 완화했다.



초기 Foot IK는 양발을 모두 바닥 기준으로 보정했기 때문에, 보행 중 들어 올려져야 하는 발까지 지면에 고정되었다.



양발의 지면 충돌 결과와 발-지면 간 높이 차이를 비교해 현재 캐릭터를 지탱하는 발을 판별하도록 변경했다.
정지 상태에서는 양발이 모두 지지발이 될 수 있도록 처리하고,
이동 중에는 양발이 모두 공중에 떠 있다고 판단되는 경우를 제외한 뒤 지면에 더 가까운 발을 지지발로 선택했다.
선택된 지지발에는 IK Weight를 1에 가깝게 적용하고, 반대쪽 발은 Weight를 0으로 줄여
원본 애니메이션의 발 들어올림 동작이 유지되도록 했다.

# Inverse Kinematics

## ❖ CODE

```
auto leftTargetPos = leftFootMatrix.r[3];
auto rightTargetPos = rightFootMatrix.r[3];

if (leftHitValid)
{
    const auto groundModel =

TransformWorldPositionToModel(XMLoadFloat3(&leftGroundHit.posi
tion), ownerTransform);

    const float targetModelY = XMVectorGetY(groundModel) +
m_footSoleOffset;
    leftTargetPos = XMVectorSetY(leftTargetPos, targetModelY);
}

if (rightHitValid)
{
    const auto groundModel =

TransformWorldPositionToModel(XMLoadFloat3(&rightGroundHit.pos
ition), ownerTransform);

    const float targetModelY = XMVectorGetY(groundModel) +
m_footSoleOffset;
    rightTargetPos = XMVectorSetY(rightTargetPos,
targetModelY);
}
```

발 목표 위치 만들기
현재 애니메이션의 발 위치에서 Y값만 지면 높이로 바꿔서 IK 목표 위치를 만듦

```
m_preIKGlobalMatrices = m_globalMatrices;

for (const auto& target : m_ikTargets)
{
    if (target.active && target.weight > 0.0f)
    {
        m_ikProcessor.SolveTwoBoneIK(
            m_globalMatrices,
            m_preIKGlobalMatrices,
            target
        );


propagateDescendantsExcept(target.rootBoneIndex,
target.midBoneIndex);

propagateDescendantsExcept(target.midBoneIndex,
target.boneIndex);
        propagateDescendants(target.boneIndex);
    }
}
```

원본 애니메이션 포즈 저장 후 IK 적용
IK 계산 시 poleVector 힌트를 원본 애니메이션으로부터 받기 위해
(자연스러운 결과 도출)
IK를 적용하기 전에 원래 애니메이션 포즈를 저장

```
float a = XMVectorGetX(XMVector3Length(XMVectorSubtract(midPos,
rootPos)));
float b = XMVectorGetX(XMVector3Length(XMVectorSubtract(endPos,
midPos)));
float c = XMVectorGetX(XMVector3Length(XMVectorSubtract(targetPos,
rootPos)));

c = std::clamp(c, std::abs(a - b) + 0.001f, a + b - 0.001f);

float cosA = (a * a + c * c - b * b) / (2.0f * a * c);
cosA = std::clamp(cosA, -1.0f, 1.0f);

XMVECTOR targetDir = XMVector3Normalize(XMVectorSubtract(targetPos,
rootPos));
XMVECTOR bendDir = XMVectorSubtract(
    preIKMidDir,
    XMVectorScale(targetDir, XMVectorGetX(XMVector3Dot(preIKMidDir,
targetDir)))
);

bendDir = XMVector3Normalize(bendDir);

const float sinA = std::sqrt(std::max(0.0f, 1.0f - cosA * cosA));

XMVECTOR newMidDir = XMVector3Normalize(
    XMVectorScale(targetDir, cosA) +
    XMVectorScale(bendDir, sinA)
);

XMVECTOR newMidPos = XMVectorAdd(rootPos, XMVectorScale(newMidDir,
a));
```

Two-Bone IK 핵심 계산
허벅지 길이 a, 종아리 길이 b, 목표 발 위치까지 거리 c를 이용해서 새 무릎 위치를 계산

발 아래 지면을 샘플링해 목표 발 위치(targetPos)를 만들고, 허벅지(root), 무릎(mid), 발(end)로 구성된 Two-Bone IK 체인에 적용
원본 애니메이션의 무릎 방향(preIKMidDir)을 기준으로 굽힘 방향(bendDir)을 계산하고, 코사인 법칙으로 새 무릎 위치(newMidPos)를 구한 뒤 본 행렬에 반영

# Inverse Kinematics

## ❖ PIX Profiling 및 성능 개선

### 문제 정의

- 기존 TrySampleVisualGround()는 발 주변 radius 내에 있는 메시를 고른 뒤 선택된 메시 내부의 삼각형을 순차 검사했다.
- 계단 메시처럼 큰 메시 하나가 후보에 포함되면 TriangleTests::Intersects가 수천~수만 회 실행되었다.
- PIX에서 FootIKComponent::OnLateUpdate 내부의 바닥 샘플링 비용이 크게 보였고 병목은 IK 계산이 아니라 메시 삼각형 raycast였다.

### 해결 방안

- 현재 Scene Map Mesh들의 삼각형을 월드 좌표로 변환해서 GroundQueryCache에 저장한다.
- XZ 평면을 일정 크기의 셀로 나누고, 각 셀에 걸치는 삼각형 번호만 gridCellsByKey에 넣는다.
- 발 IK가 바닥을 찾을 때는 발 위치가 속한 셀만 찾고, 그 셀의 삼각형들만 Ray 검사한다.

```
struct GroundQueryTriangle
{
    DirectX::XMFLOAT3 v0 = {};
    DirectX::XMFLOAT3 v1 = {};
    DirectX::XMFLOAT3 v2 = {};
    DirectX::XMFLOAT3 normal = {0.0f, 1.0f, 0.0f};
    const GameObject* sourceObject = nullptr;
    const MeshResource* sourceMesh = nullptr;
};

struct GroundQueryCell
{
    std::vector<size_t> triangleIndexesInThisCell;
};

struct GroundQueryCache
{
    const Scene* scene = nullptr;
    size_t sourceMeshCount = 0;
    std::vector<GroundQueryTriangle> triangles;
    std::unordered_map<std::int64_t, GroundQueryCell>
gridCellsByKey;
    bool isValid = false;
};
```

GroundQueryCache 구성
맵 메시의 삼각형을 월드 좌표 기준으로 변환해 캐시에 저장

```
const size_t triangleIndex = cache.triangles.size();
cache.triangles.push_back(triangle);

const std::int32_t minCellX =
ToGroundQueryCellCoord(minX);
const std::int32_t maxCellX =
ToGroundQueryCellCoord(maxX);
const std::int32_t minCellZ =
ToGroundQueryCellCoord(minZ);
const std::int32_t maxCellZ =
ToGroundQueryCellCoord(maxZ);

for (std::int32_t cellZ = minCellZ; cellZ <= maxCellZ;
++cellZ)
{
    for (std::int32_t cellX = minCellX; cellX <=
maxCellX; ++cellX)
    {
        const auto cellKey =
MakeGroundQueryCellKey(cellX, cellZ);

cache.gridCellsByKey[cellKey].triangleIndexesInThisCe
ll.push_back(triangleIndex);
    }
}
```

XZ 그리드 셀 등록
삼각형의 XZ 최소/최대 범위를 기준으로 겹치는 셀을 계산하고 각 셀에 삼각형 번호만 저장

```
const auto queryCellKey = MakeGroundQueryCellKey(queryCellX, queryCellZ);

if (auto it = groundCache->gridCellsByKey.find(queryCellKey);
    it != groundCache->gridCellsByKey.end())
{
    queryCell = &it->second;
}

for (const size_t triangleIndex : queryCell->triangleIndexesInThisCell)
{
    const auto& cachedTriangle = groundCache->triangles[triangleIndex];

    const auto cachedV0 = DirectX::XMLoadFloat3(&cachedTriangle.v0);
    const auto cachedV1 = DirectX::XMLoadFloat3(&cachedTriangle.v1);
    const auto cachedV2 = DirectX::XMLoadFloat3(&cachedTriangle.v2);

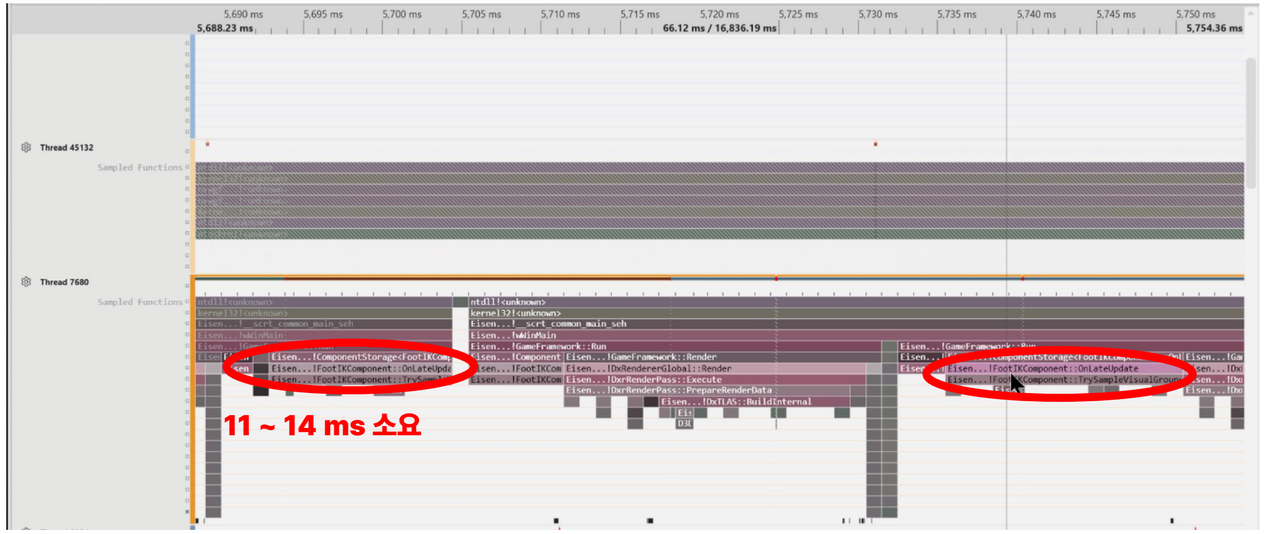
    ++triangleTestCount;
    if (!DirectX::TriangleTests::Intersects(rayOrigin, rayDir, cachedV0,
cachedV1, cachedV2, distance))
    {
        continue;
    }
}
```

캐시 기반 바닥 조회
런타임에는 발 위치가 속한 셀만 조회해 전체 메시 대신 가까운 삼각형 후보만 Ray 검사

# Inverse Kinematics

## ❖ PIX Profiling 및 성능 개선

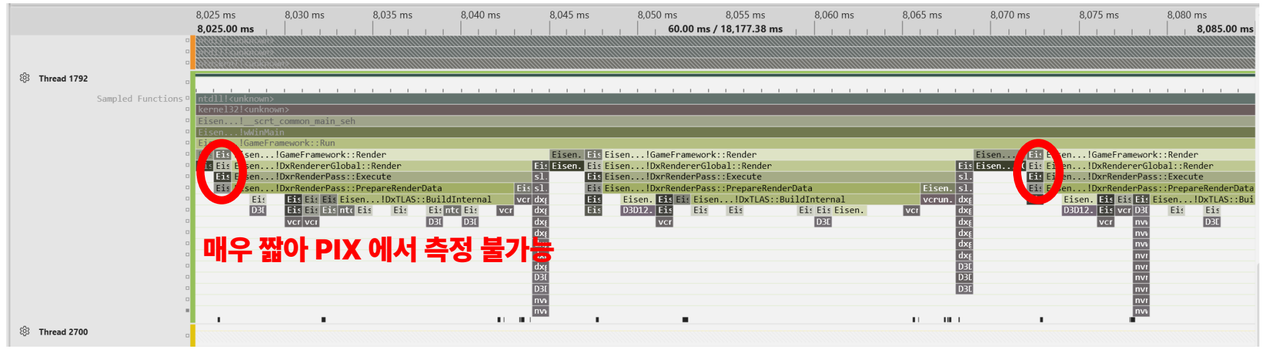
**[개선 전]**




**17,772**

기존 삼각형 검사 수

**[개선 후]**




**32~485**

캐시 적용 후 삼각형 검사 수

EisenValor

# 공격-피격 타이밍 동기화

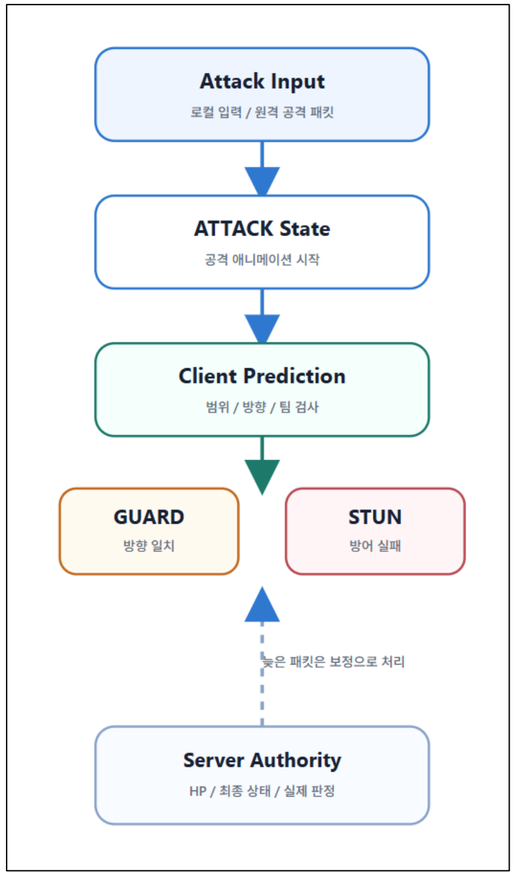
## 개요

**무엇을 구현했나**

공격자가 `ATTACK` 상태에 들어간 순간, 클라이언트가 먼저 맞을 대상을 찾고 상대의 피격 반응을 재생하는 전투 동기화 구조를 정리했다.

**왜 필요했나**

서버 패킷을 기다린 뒤 피격 애니메이션을 재생하면 공격과 피격 타이밍이 어긋난다. 따라서 화면 반응은 클라이언트가 빠르게 보여주고, HP와 최종 판정은 서버가 확정하도록 역할을 나눴다.




- 공격 상태를 선딜(PRE_DELAY), 공격(ATTACK), 후딜(POST_DELAY)로 분리하고,
실제 타격 구간인 ATTACK state 진입 시점에 클라이언트가 공격 범위를 선검사하도록 구현했다.
이로써 서버 피격 패킷을 기다린 뒤 STUN을 재생하던 방식에서 발생하던 공격-피격 타이밍 지연을 줄였다.
- ATTACK 진입 시 서버와 같은 기준을 사용하여 공격자의 위치와 방향을 기준으로 공격 범위를 계산하고,
범위 안의 적 대상을 찾은 뒤 공격 방향을 비교한다.
공격 방향이 일치하면 GUARD를, 방어가 실패하면 target FSM에 attackType과 attackDir를 저장한 뒤 STUN을 즉시 요청한다.
- 이후 서버 패킷은 HP 감소와 최종 상태를 확정하는 보정 역할로 처리했다.



[사진 1] 상대가 공격을 방어했을 때



[사진 2] 공격 성공 시 바로 피격 액션 재생

## ❖ 문제와 해결

⚠ 예측 STUN 후 서버 STUN이 오면 피격 애니메이션이나 사운드가 중복 재생될 수 있다.

클라이언트 예측은 플레이어가 즉시 체감하는 화면 반응을 담당하고, 서버 패킷은 실제 전투 결과를 확정하는 권한 보정으로 분리했다.
클라이언트는 애니메이션, 사운드 같은 시청각적 반응을 즉시 재생한다.
HP 감소, 최종 상태, 실제 피격 성공 여부는 서버 판정을 기준으로 처리해 클라이언트 예측이 게임 결과를 직접 결정하지 않도록 했다.

# 공격-피격 타이밍 동기화

## ❖ CODE

```
AttackTiming
GetAttackTiming(GENERAL_ATTACK_TYPE type)
{
    switch (type)
    {
    case GENERAL_ATTACK_TYPE_HEAVY:
        return {21.0f / 30.0f, 6.0f /
30.0f, 12.0f / 30.0f};
    case GENERAL_ATTACK_TYPE_AREA:
        return {30.0f / 30.0f, 6.0f /
30.0f, 21.0f / 30.0f};
    default:
        return {9.0f / 30.0f, 3.0f /
30.0f, 3.0f / 30.0f};
    }
}
```

공격을 PRE_DELAY, ATTACK, POST_DELAY로 분리하고, 실제 타격 판정은 ATTACK state에 진입한 시점에만 수행한다.

```
AttackShape GetAttackShape(GENERAL_ATTACK_TYPE type)
{
    switch (type)
    {
    case GENERAL_ATTACK_TYPE_AREA:
        return {7.0f, 160.0f};
    default:
        return {7.0f, 120.0f};
    }
}

const AttackShape shape = GetAttackShape(type);
GameObject* target = FindPredictedAttackTarget(scene,
obj, shape.radius, shape.degree);
```

ATTACK state 진입 시 공격 타입별 부채꼴 범위를 가져오고 클라이언트가 먼저 범위 안의 target을 찾는다.

```
if (defenderGuarded || attackerGuarded)
{
    targetFsm->RequestState(FSMComponent::StateRequestType::Guard);
    fsm->RequestState(FSMComponent::StateRequestType::Guard);
    return;
}

targetFsm->SetHitReact(fsm->GetCurAttackType(), attackerDir);
targetFsm->RequestState(FSMComponent::StateRequestType::Stun);
```

공격 방향과 방어 방향이 일치하면 GUARD를 먼저 재생하고, 방어가 실패하면 target FSM에 피격용 방향과 공격 타입을 저장한 뒤 STUN을 재생한다.

Source: StateImplementations.cpp / GeneralAttackState::Update()

# Combat Input Architecture

## 개요

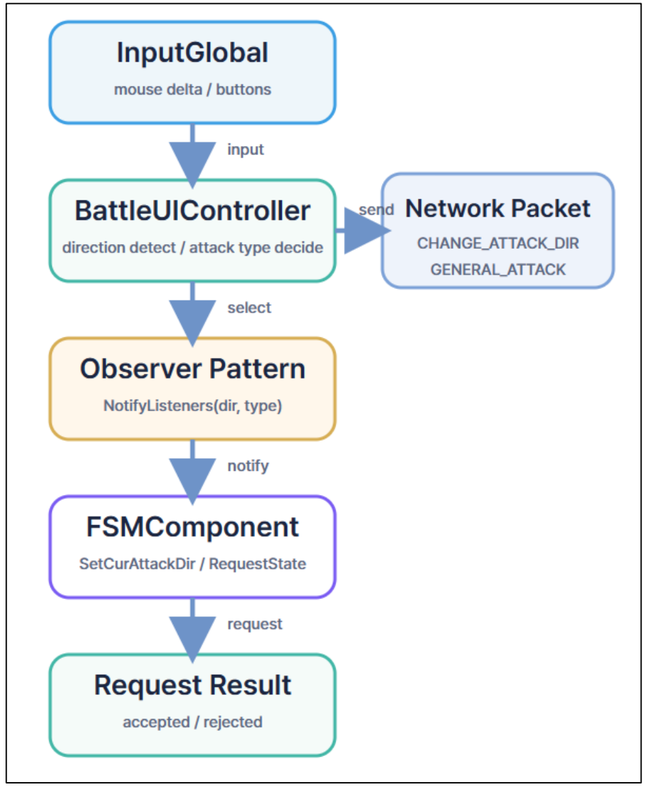
### 무엇을 구현했는가

전투 자세에서 플레이어 머리 위에 3방향 공격 UI를 띄우고, 마우스 이동으로 방향을 고른 뒤 좌클릭, 우클릭, 동시 입력으로 약공격, 강공격, 범위공격을 확정하는 구조를 구현했다.

### 왜 이러한 UI 구조가 필요했는가

전투 입력은 단순 버튼 입력이 아니라 방향 선택, 공격 종류 선택, FSM 상태 요청, 서버 패킷 전송, 애니메이션 키 선택이 한 번에 맞물려야 한다. 이 흐름을 분리하지 않으면 입력 지연, 중복 공격, 방향 불일치가 발생한다.

- InputGlobal이 먼저 마우스 이동량과 버튼 입력을 수집하고, 이를 전투 입력 의도로 넘긴다.
- 이후 BattleUIControllerComponent가 마우스 누적 이동을 기준으로 위 / 왼쪽 / 오른쪽 공격 방향을 판정하고,
  좌클릭 / 우클릭 / 동시 클릭 / 휠 클릭으로 약공격 / 강공격 / 범위공격 / 무장해제 공격 타입을 확정한다. 방향 또는 공격 타입이 바뀌면 NotifyListeners(dir, type)를 호출한다.
- NotifyListeners는 선택된 공격 방향과 공격 타입을 등록된 두 개의 Listener들에게 전파한다.
  하나는 FSM의 CurAttackDir을 갱신하고, 다른 하나는 3방향 UI 버튼과 이미지 상태를 갱신한다.
- 동시에 BattleUIControllerComponent는 방향 변경 시 CHANGE_ATTACK_DIR 패킷을 보내고 공격 확정 시 GENERAL_ATTACK 패킷을 서버로 전송한다.
  FSM 요청 결과가 거절되면 입력 버퍼만 초기화하고, 수락되면 UI / FSM / 서버 패킷이 같은 방향과 공격 타입을 공유한 상태로 전투 입력을 처리한다.










[사진] 공격 방향, 타입, 애니메이션이 UI와 연동되고 있는 인게임 화면 캡쳐

# Combat Input Architecture

## ❖ 문제와 해결



BattleUI가 방향 선택, 공격 타입 판정, 현재 모드, 공격 실행 가능 여부, 애니메이션까지 모두 직접 관리하여 UI 표시 상태와 FSM 전투 상태가 어긋난다.



**판정과 실행 분리**
BattleUIControllerComponent가 마우스 입력을 분석해 공격 방향과 공격 타입을 판정하고 그 결과를 FSMComponent에 전달한다.
FSMComponent는 전달받은 방향과 타입을 현재 전투 상태 값으로 저장하고, RequestState()를 통해 해당 공격을 실행할 수 있는지 판단한다.



UI 리스너가 컴포넌트 포인터를 직접 잡고 있어 Scene 갱신이나 오브젝트 재생성 뒤 잘못된 FSM을 참조한다.



**Engine 핵심 설계 원리에 맞게 UI 동작 구조 수정**
포인터가 아닌 ownerHandle을 저장하고, 콜백 실행 시 활성 씬에서 오브젝트를 다시 찾아 FSMComponent를 얻도록 했다.

# Combat Input Architecture

## ❖ CODE

```
auto& input = GLOBAL(InputGlobal);
DX::XMFLOAT2 mouseDelta =
input.GetMouseDelta();

GENERAL_ATTACK_DIR_TYPE detectedDir =

CalculateGuardDirection(m_accumulatedDel
taX, m_accumulatedDeltaY);

bool isLeftPressed =
input.GetInput(VK_LBUTTON);
bool isRightPressed =
input.GetInput(VK_RBUTTON);
bool isMiddlePressed =
input.GetInput(VK_MBUTTON);

std::optional<GENERAL_ATTACK_TYPE>
currentType = std::nullopt;

if (isMiddlePressed)
    currentType =
GENERAL_ATTACK_TYPE_DISARM;
else if (isLeftPressed && isRightPressed)
    currentType =
GENERAL_ATTACK_TYPE_AREA;
else if (isRightPressed)
    currentType =
GENERAL_ATTACK_TYPE_HEAVY;
else if (isLeftPressed)
    currentType =
GENERAL_ATTACK_TYPE_LIGHT;
```

마우스 이동량을 누적해 공격 방향을 TOP / LEFT / RIGHT로 판정하고,
마우스 버튼 조합으로 약공격 / 강공격 / 범위공격 / 무장해제 타입을 결정한다.
공격 중에는 방향 변경 입력을 막아 공격 상태 도중 UI 방향이 흔들리지 않도록 한다.

```
void
BattleUIControllerComponent::UpdateUI
Selection(
    GENERAL_ATTACK_DIR_TYPE
selectedDir,

std::optional<GENERAL_ATTACK_TYPE>
attackType
)
{
    m_currentSelectedDir =
selectedDir;
    m_currentAttackType = attackType;

    NotifyListeners(selectedDir,
attackType);
}
```

선택된 방향이나 공격 타입이 바뀌면 UpdateUISelection을 통해 현재 UI 상태를 갱신하고,
NotifyListeners를 호출해 등록된 listener들에게 변경 내용을 전파한다.

```
AddListener(
    ownerHandle,
    [ownerHandle](GENERAL_ATTACK_DIR_TYPE dir,

std::optional<GENERAL_ATTACK_TYPE> type)
    {
        auto* scene =
GLOBAL(SceneGlobal).GetActiveScene();
        if (!scene) return;

        if (auto* ownerObj = scene-
>TryGetGameObject(ownerHandle))
        {
            if (auto* currentFsm = ownerObj-
>GetComponent<FSMComponent>())
            {
                currentFsm-
>SetCurAttackDir(static_cast<uint8_t>(dir));
            }
        }
    }
);
```

첫 번째 listener는 BattleUI에서 선택한 공격 방향을 FSMComponent의 CurAttackDir에 반영한다.
UI에서 선택한 방향과 FSM이 사용하는 공격 방향이 같은 값으로 유지되게 한다.

```
AddListener(
    m_uiRootObjHandle,
    [=](GENERAL_ATTACK_DIR_TYPE dir,

std::optional<GENERAL_ATTACK_TYPE>
type)
    {
        updateBtn(upBtn, upImg, dir
== GENERAL_ATTACK_DIR_TYPE_TOP);
        updateBtn(leftBtn, leftImg,
dir == GENERAL_ATTACK_DIR_TYPE_LEFT);
        updateBtn(rightBtn, rightImg,
dir == GENERAL_ATTACK_DIR_TYPE_RIGHT);
    }
);
```

두 번째 listener는 같은 방향 값을 사용해 UpUI, LeftUI, RightUI의 선택 상태를 갱신한다. FSM과 화면 UI를 동기화한다.

```
if (!fsm->RequestState(attackRequest))
{
    m_pendingLeftClick = false;
    m_pendingRightClick = false;
    m_inputBufferTimer = 0.0f;
    return;
}

FB_STRUCTS::GeneralAttackInfo
attackInfo(finalType, m_currentSelectedDir);
auto pb =
NetBridge::C2S::Make_CS_GENERAL_ATTACK_PACKET(&
attackInfo);
GLOBAL(NetBridge::NetworkGlobal).Send(std::move
(pb));

fsm-
>SetCurAttackDir(static_cast<uint8_t>(m_current
SelectedDir));
```

공격 타입이 확정되면 FSM에 공격 상태 전이를 요청한다.
FSM 요청이 거절되면 입력 버퍼만 초기화하고, 수락되면 방향과 공격 타입을 GENERAL_ATTACK 패킷에 담아 서버로 전송한다.

Source: BattleUIControllerComponent.cpp

EisenValor

# Finite State Machine

## 개요

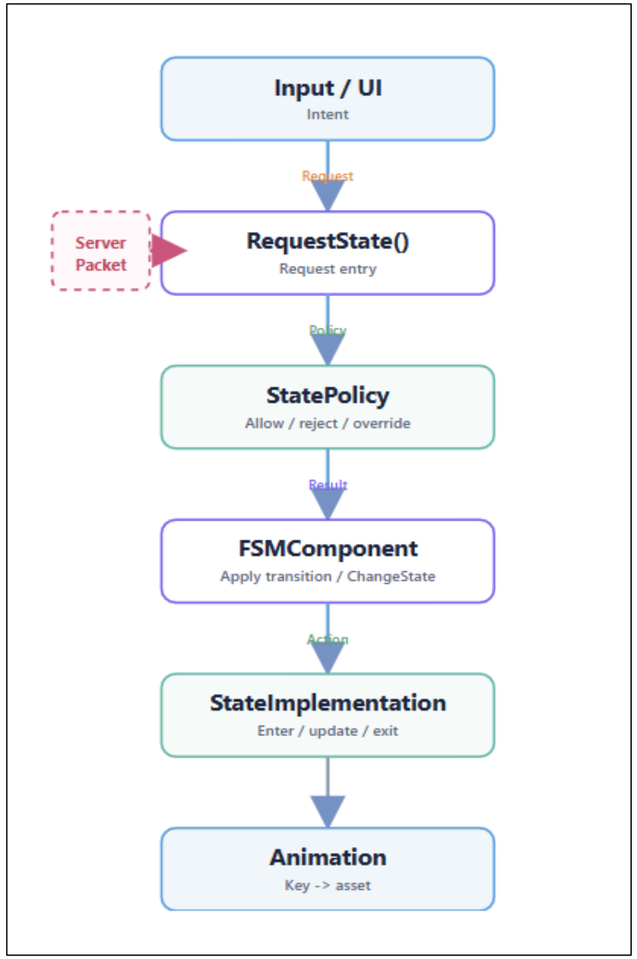
### 무엇을 구현했나

입력 코드가 바로 상태를 바꾸지 않고, 모든 전이를
`RequestState()` 로 모아 `StatePolicy` 가 판정한 뒤
`ChangeState()` 를 수행하는 정책 기반 FSM을 구현했다.

### 왜 필요했나

전투 액션 게임은 입력 반응성, 공격 모션 일관성,
서버 보정 수용을 동시에 만족해야 한다. 상태 전이 규칙이 분산되면
이 세 조건이 쉽게 충돌한다.

| | |
|---|---|
| 1 | **요청 기반 진입점으로 통합**<br>이동, 공격, 구르기, 스턴, 서버 보정은 먼저 RequestState로 들어간다.<br>이 단계에서 플레이어와 병사 객체를 분리하고, 병사는 ForcedServerCorrection만 허용한다. |
| 2 | **정책 객체가 전이 가능 여부를 판정**<br>StatePolicy에서 현재 상태와 요청 종류를 보고 전이를 허용, 거절, 강제 보정 중 하나로 결정한다.<br>공격 중에는 이동과 서버 보정을 막아 전투 흐름을 보호한다. |
| 3 | **상태 실행과 애니메이션을 분리**<br>StatePool에서 상태 객체를 보관하고, StateImplementation이 실제 동작을 담당한다.<br>StateImplementation에서는 애니메이션 키를 계산하고 타이머를 누적해 다음 단계로 넘긴다. |
| 4 | **공격은 3단계 시간 모델로 분해**<br>AttackTiming 구조체가 공격별 PreDelay / Attack / PostDelay를 분리한다.<br>선딜레이, 판정 구간, 후딜레이를 제어해 조작감과 네트워크 보정 충돌을 줄인다. |
| 5 | **Animation 단계에서 최종 키 재생 및 서버 상태와 수렴** |



EisenValor

# Finite State Machine

## ❖ 문제와 해결

이동, 공격, 피격, 서버 보정이 각 코드에서 직접 상태를 바꾸면 예외 처리가 분산되고 충돌 원인을 찾기 어렵다.

모든 전이를 RequestState() 하나로 모으고, 실제 판단은 StatePolicy에 집중시켰다.

공격 선딜과 후딜 중 다른 요청이 섞이면 모션이 끊기거나 공격 판정 흐름이 깨진다.

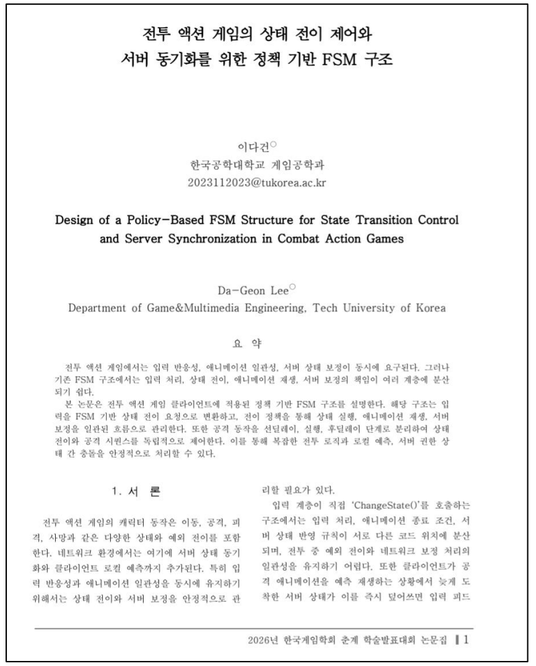
PRE_DELAY, ATTACK, POST_DELAY를 공격 시퀀스로 묶고,
StatePolicy가 이 구간에서 이동과 서버 보정을 거절하게 했다

공격별로 선딜레이와 후딜레이 시간이 다르다.
단일 ATTACK 상태만 사용하면 공격 종류별 시간 구간을 독립적으로 제어할 수 없다.

AttackTiming 구조체로 공격 유형마다 서로 다른 시간 파라미터를 적용했다.

## ❖ 논문 작성 및 포스터 제작

**2026 한국게임학회 춘계 학술발표대회 포스터 발표**



전투 액션 게임의 상태 전이 제어와
서버 동기화를 위한 정책 기반 FSM 구조

이다건
한국공학대학교 게임공학과
2023112023@tukorea.ac.kr

Design of a Policy-Based FSM Structure for State Transition Control and Server Synchronization in Combat Action Games

Da-Geon Lee
Department of Game&Multimedia Engineering, Tech University of Korea

요 약

전투 액션 게임에서는 입력 반응성, 애니메이션 일관성, 서버 상태 보정이 동시에 요구된다. 그러나 기존 FSM 구조에서는 입력 처리, 상태 전이, 애니메이션 재생, 서버 보정의 책임이 여러 계층에 분산되기 쉽다.

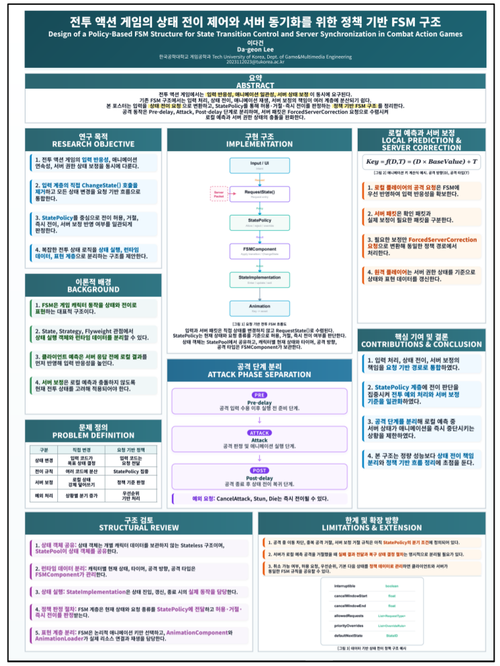
본 논문은 전투 액션 게임 클라이언트에 적용된 정책 기반 FSM 구조를 설명한다. 해당 구조는 입력을 FSM 기반 상태 전이 요청으로 변환하고, 전이 정책을 통해 상태 실행, 애니메이션 재생, 서버 보정을 일관된 흐름으로 관리한다. 또한 공격 동작을 선딜레이, 실행, 후딜레이 단계로 분리하여 상태 전이와 공격 시퀀스를 독립적으로 제어한다. 이를 통해 복잡한 전투 로직과 로컬 예측, 서버 권한 상태 간 충돌을 안정적으로 처리할 수 있다.

### 1. 서 론

전투 액션 게임의 캐릭터 동작은 이동, 공격, 피격, 사망과 같은 다양한 상태와 예외 전이를 포함한다. 네트워크 환경에서는 여기에 서버 상태 동기화와 클라이언트 로컬 예측까지 추가된다. 특히 입력 반응성과 애니메이션 일관성을 동시에 유지하기 위해서는 상태 전이와 서버 보정을 안정적으로 관리할 필요가 있다.

입력 계층이 직접 'ChangeState()'를 호출하는 구조에서는 입력 처리, 애니메이션 종료 조건, 서버 상태 반영 규칙이 서로 다른 코드 위치에 분산되며, 전투 중 예외 전이와 네트워크 보정 처리의 일관성을 유지하기 어렵다. 또한 클라이언트가 공격 애니메이션을 예측 재생하는 상황에서 늦게 도착한 서버 상태가 이를 즉시 덮어쓰면 입력 피드





전투 액션 게임의 상태 전이 제어와 서버 동기화를 위한 정책 기반 FSM 구조

Design of a Policy-Based FSM Structure for State Transition Control and Server Synchronization in Combat Action Games

이다건
한국공학대학교 게임공학과
2023112023@tukorea.ac.kr

요약 ABSTRACT

연구 목적 RESEARCH OBJECTIVE

구현 구조 IMPLEMENTATION

로컬 예측과 서버 보정 LOCAL PREDICTION & SERVER CORRECTION

이론적 배경 BACKGROUND

공격 단계 분리 ATTACK PHASE SEPARATION

문제 정의 PROBLEM DEFINITION

핵심 기여 및 결론 CONTRIBUTIONS & CONCLUSION

구조 검토 STRUCTURAL REVIEW

한계 및 확장 방향 LIMITATIONS & EXTENSION

https://dageonlee.tistory.com/1

# Finite State Machine

## ❖ CODE

```
enum class StateRequestType : uint8_t {
    Move,
    StopMove,
    AttackLight,
    AttackHeavy,
    CancelAttack,
    Stun,
    Die,
    IdleRecovery,
    Guard,
    ForcedServerCorrection
};

struct StateTransitionDecision {
    bool accepted = false;
    uint8_t nextStateType = 0;
    bool ignoreExitTime = false;
};

class IStatePolicy {
public:
    virtual StateTransitionDecision Resolve(
        const FSMComponent& fsm,
        StateRequestType request,
        uint8_t targetStateOverride
    ) const = 0;
};
```

요청/정책 구조

- StateRequestType:  입력이나 서버 보정을 바로 상태 변경으로 처리하지 않고, 먼저 “상태 전이 요청”으로 표현하기 위한 enum
- StateTransitionDecision: StatePolicy가 요청을 판단한 결과 accepted가 true이면 전이를 허용

```
StateTransitionDecision PlayerStatePolicy::Resolve(
    const FSMComponent& fsm,
    StateRequestType request,
    uint8_t targetStateOverride
) const
{
    switch (request)
    {
    case StateRequestType::Move:
        if (IsPlayerMovementBlockedState(fsm.GetCurStateType()))
            return Reject();

        return Accept(targetStateOverride != 0
            ? targetStateOverride
            : static_cast<uint8_t>(FB_ENUMS::PLAYER_STATE_TYPE_WALK));

    case StateRequestType::AttackLight:
    case StateRequestType::AttackHeavy:
        if (IsPlayerAttackSequenceState(fsm.GetCurStateType()))
            return Reject();

        return Accept(static_cast<uint8_t>(FB_ENUMS::PLAYER_STATE_TYPE_PRE_DELAY));

    case StateRequestType::CancelAttack:
        return Accept(static_cast<uint8_t>(FB_ENUMS::PLAYER_STATE_TYPE_IDLE), true);

    case StateRequestType::Stun:
        return Accept(static_cast<uint8_t>(FB_ENUMS::PLAYER_STATE_TYPE_STUN), true);

    case StateRequestType::ForcedServerCorrection:
        if (IsPlayerAttackSequenceState(fsm.GetCurStateType()))
            return Reject();

        return Accept(targetStateOverride, true);

    default:
        return Reject();
    }
}
```

정책 전이 가능 여부 판단

- PlayerStatePolicy::Resolve(): 현재 FSM 상태와 요청 종류를 보고 전이를 허용할지 판단

```
bool FSMComponent::RequestState(StateRequestType request,
uint8_t targetStateOverride)
{
    static const PlayerStatePolicy playerPolicy;

    StateTransitionDecision decision =
        playerPolicy.Resolve(*this, request,
targetStateOverride);

    if (!decision.accepted)
        return false;

    switch (request)
    {
    case StateRequestType::AttackLight:

SetCurAttackType(static_cast<uint8_t>(FB_ENUMS::GENERAL_ATTAC
K_TYPE_LIGHT));
        break;

    case StateRequestType::AttackHeavy:

SetCurAttackType(static_cast<uint8_t>(FB_ENUMS::GENERAL_ATTAC
K_TYPE_HEAVY));
        break;

    default:
        break;
    }

    ChangeState(decision.nextStateType,
decision.ignoreExitTime);
    return true;
}
```

FSMComponent 에서 정책 결과를 실행

- 정책이 허용한 경우 공격 요청인지 확인하여 SetCurAttackType()으로 현재 공격 타입 저장
- ChangeState()을 호출하여 StatePolicy가 만든 decision 결과 실행

# Helicopter Simulation



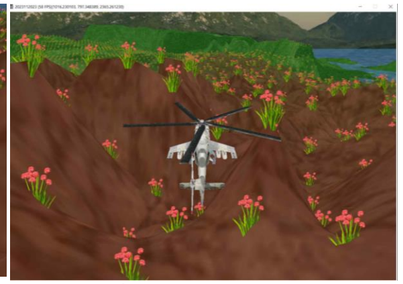
Sparkle



Motion Blur



GPU Billboard
Instancing Rendering



Tessellation

| | |
|---|---|
| 프로젝트 이름 | Helicopter Simulation |
| 장르 | 3D 시뮬레이션 |
| 설명 | 3D 헬리콥터 전투 데모 |
| 개발 인원 | 1명 |
| 담당 역할 | Height Map 기반 Terrain, Water Rendering,<br>GPU Instancing Billboard Rendering, Tessellation<br>Shadow Map 기반 그림자, Motion Blur, Sparkle Effect |
| 개발 언어 | C++20, HLSL |
| 사용 툴 & 라이브러리 | VS 2022, C++20, DirectX12, Git |
| 제작 기간 | 2025.10 ~ 2025.12 |

# Helicopter Simulation



## Motion Blur

### 개요

**무엇을 만들었나**

장면을 한 번 렌더링한 뒤, 현재 프레임 텍스처와 이전 프레임 히스토리 버퍼를 Compute Shader에서 섞어 최종 화면을 만든다. 이로 인해 플레이어가 빠르게 움직일 때 잔상이 생긴다.

**왜 필요한가**

플레이어가 빠르게 이동할 때 이전 프레임의 색을 일부 남겨 화면에 잔상을 만들면 속도감이 분명하게 보인다.

- Compute Shader 기반 후처리로 구현했다. 먼저 장면을 Swap Chain Back Buffer에 직접 렌더링하지 않고, 중간 렌더 타깃인 IntermediateRenderTarget에 렌더링한다. 이를 통해 현재 프레임을 텍스처(SRV)로 사용할 수 있으며, 이전 프레임을 저장한 HistoryBuffer와 함께 Compute Shader의 입력으로 활용한다.
- 렌더링이 끝나면 플레이어의 현재 속도를 가져와 최대 속도 기준으로 정규화하고, 이를 이용해 BlendFactor를 계산한다. BlendFactor는 현재 프레임과 이전 프레임을 혼합하는 비율을 결정하는 값으로, 플레이어가 빠르게 이동할수록 이전 프레임의 비중이 커져 더 강한 Motion Blur 효과가 나타난다.
- Compute Shader에서 현재 프레임과 이전 프레임을 블렌딩한 뒤 결과를 출력 텍스처에 기록한다. 첫 프레임에는 이전 프레임 데이터가 없으므로 중간 렌더 타깃을 히스토리 버퍼에 복사해 초기화한다. 이후 프레임부터는 입력 리소스를 SHADER_RESOURCE 상태로, 출력 텍스처를 UNORDERED_ACCESS 상태로 전환한 뒤 8×8 스레드 그룹 단위로 Dispatch를 수행한다.
- 연산이 완료되면 결과를 Back Buffer에 복사하여 화면에 출력하고, 동시에 히스토리 버퍼를 갱신하여 다음 프레임의 Motion Blur 계산에 사용한다.

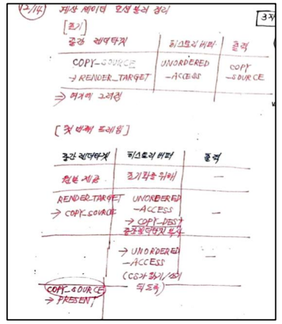


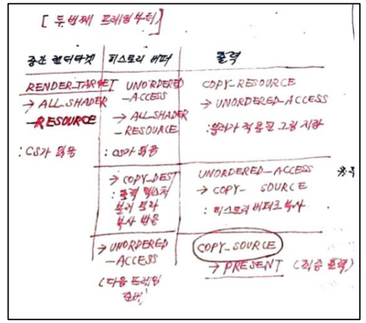


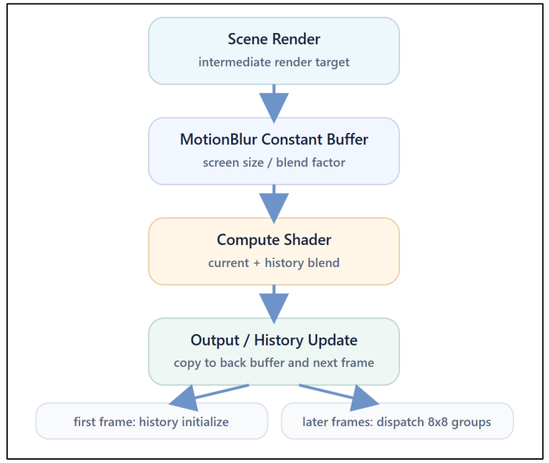
### ❖ 문제와 해결

메인 렌더링 함수에서 텍스처들의 Resource State 전환 순서를 맞추는 과정이 어려웠다.
현재 프레임, 이전 프레임, 출력 텍스처가 각각 읽기, 쓰기, 복사용 상태를 오가기 때문에 전환 순서가 복잡했고, 잘못 설정하면 렌더링 오류가 발생했다.

종이에 표를 그려, 직접 각각의 Resource State 변환 과정을 써 가면서 해결했다







# Helicopter Simulation

## Tessellation

### 개요

**무엇을 구현했나**

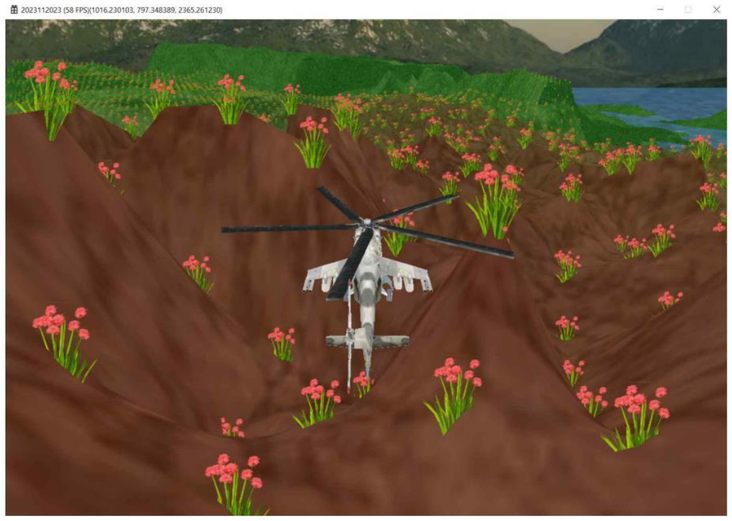
Terrain에 Tessellation LOD를 적용했다. 가까운 지형은 더 많이 나누어 울퉁불퉁한 높이 변화가 잘 보이게 하고, 먼 지형은 적게 나누어 불필요한 계산을 줄이는 구조다.

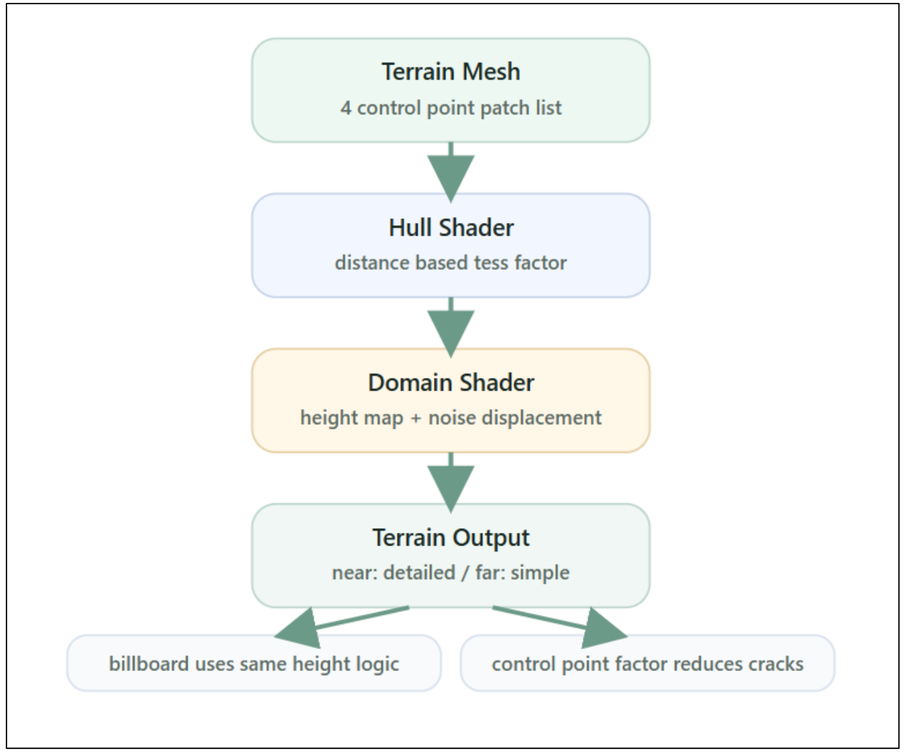
**왜 필요한가**

기본 Terrain Mesh를 항상 촘촘하게 만들면 먼 지형까지 많은 정점과 삼각형을 처리해야 해서 프레임레이트가 떨어질 수 있다. Tessellation LOD를 사용하면 가까운 지형만 더 세밀하게 나누고 먼 지형은 단순하게 유지해, 지형 품질과 성능을 함께 관리할 수 있다.

- Terrain Shader 파이프라인에 Hull Shader와 Domain Shader를 추가하고, Terrain Mesh의 입력 토폴로지를 D3D_PRIMITIVE_TOPOLOGY_4_CONTROL_POINT_PATCHLIST로 바꾸었다.
- Hull Shader는 패치 중심과 카메라 사이의 거리를 기준으로 Tessellation Factor를 계산한다.
- Domain Shader는 각 Control Point의 거리, Height Map, Noise 값을 이용해 최종 지형 높이를 만든다.
- 가까운 지형은 MaxTessellationFactor에 가까워져 더 촘촘해지고, 먼 지형은 MinTessellationFactor에 가까워져 단순하게 렌더링된다.



# TinyMMORPG




| | |
|---|---|
| 프로젝트 이름 | TinyMMORPG |
| 장르 | 2D MMORPG |
| 설명 | IOCP 기반 비동기 서버 구조로 구현한<br>2D MMORPG 몬스터 처치 게임 |
| 개발 인원 | 1명 |
| 담당 역할 | IOCP 네트워크 처리, NPC/A* AI, Lua 보스 AI 연동, Section 처리,<br>DB 연동, 클라이언트 연동 및 StressTest |
| 개발 언어 | C++20 |
| 사용 툴 & 라이브러리 | VS 2022, IOCP, SFML 2.5.1, Intel TBB, Lua, ODBC, MS-SQL, Git |
| 제작 기간 | 2026.05 ~ 2026.06 |

# TinyMMORPG

C++ | Windows IOCP | AcceptEx | A* Path Finding | Lua C API | ODBC / MS-SQL | Stress Test

## ❖ 서버 처리 흐름

**Client**
로그인, 이동, 전투, 아이템 패킷 송신

**IOCP**
AcceptEx, WSARecv, WSASend 완료 이벤트 수신

**Worker thread**
GQCS로 네트워크 완료, NPC 이동 이벤트를 꺼내 처리

**World State**
좌표, HP, 장비, 스킬, 퀘스트 상태 갱신

**Section(Sector)**
섹터 주변 오브젝트만 수집해 Broadcast 범위 축소

**DB / Lua**
플레이어 저장과 보스 판단 로직 연동

### 핵심 구현 포인트

#### ● IOCP 네트워크 처리

소켓을 IOCP에 연결하고, 워커스레드가 GQCS로 완료 이벤트를 받아 접속, 수신, 송신, NPC 이동 이벤트 처리

- AcceptEx 기반 비동기 접속
- 패킷 조립용 remain buffer 처리
- WakeUpNpc가 timer_queue에 이동 예약을 넣고 TimerThread가 PostQueuedCompletionStatus로 IOCP 큐에 전달
- OP_NPCMOVE 이벤트를 GQCS 워커스레드가 꺼내 how_to_npc_moves 실행
- hardware_concurrency 기준 워커스레드 생성

#### ● NPC / A* AI

NPC는 주변 플레이어가 보일 때 활성화되고, Agro 상태에서는 장애물 타일을 피해 A* 경로를 계산해 한 칸씩 추적.

- 8방향 탐색
- 맨해튼 거리 휴리스틱
- SEARCH_LIMIT로 서버 연산량 제한

#### ● Lua 보스 AI

보스의 HP 비율과 패턴 쿨타임을 Lua 함수에 전달하고, blink, pattern, berserk 결과를 받아 서버 행동으로 반영

- boss_guardian.lua 분리
- HP 50% 이하 berserk 판단
- 패턴 쿨타임 동적 변경

#### ● Section(Sector) 시야 처리

전체 월드를 20타일 단위 섹터로 나누고, 이동 시 섹터 등록을 갱신해 가까운 대상에게만 상태 변화 전송

- get_sector_x / get_sector_y
- add_to_sector / remove_from_sector
- collect_near_objects로 관심 영역 수집

#### ● DB 연동

ODBC DSN으로 MS-SQL에 연결하고, 일반 플레이어의 로그인 조회와 종료 저장을 통해 위치, 성장, 장비, 스킬, 퀘스트 정보를 유지

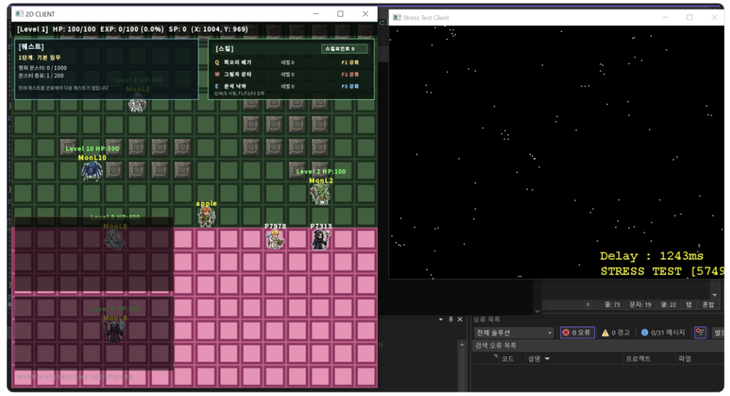
- 2026_GameServer_ODBC 연결
- sp_user_login 프로시저
- sp_user_save_pos_stats 저장

#### ● 클라이언트 / StressTest

SFML 클라이언트는 서버 패킷을 받아 캐릭터, 몬스터, UI 상태를 갱신하고, StressTest는 DB 저장 부하와 분리된 더미 계정으로 접속과 이동 패킷 부하를 확인

- client.cpp 패킷 처리
- 더미 계정 P0, P1... 자동 로그인
- 무작위 이동 패킷 송신

**StressTest 기록**




**15,000**
MAX_PLAYERS 기준 동접 설계

**200,000**
MAX_NPCS 기준 NPC 풀

# Zombie Shooting Game







| | |
|---|---|
| 프로젝트 이름 | Zombie Shooting Game |
| 장르 | 3D Shooting Game |
| 설명 | Unreal Engine 5의 Blueprint Visual Scripting으로 구현한<br>3D 좀비 슈팅 게임 |
| 개발 인원 | 1명 |
| 담당 역할 | 플레이어 이동 및 사격 시스템, 좀비 AI와 행동 트리, 애니메이션 블렌드 스페이스 및 몽타주, 체력·타이틀 UI, 랜드스케이프 기반 맵 구성 |
| 개발 언어 | Blueprint Visual Scripting |
| 사용 툴 & 라이브러리 | Unreal Engine 5 |
| 제작 기간 | 2025.05 ~ 2025.06 |

# NightMare Adventure







| | |
|---|---|
| 프로젝트 이름 | NightMareAdventure |
| 장르 | 2D 생존 액션 게임 |
| 설명 | 몬스터를 피하고 아이템을 활용하여<br>제한 시간 동안 생존하는 2스테이지 2D 게임 |
| 개발 인원 | 1명 |
| 담당 역할 | 플레이어 이동 및 조준, 몬스터 스폰, 폭탄/포션/열쇠 상호작용,<br>충돌 처리, 체력 UI, 스테이지 전환, 승패 화면, 사운드 재생 등 |
| 개발 언어 | Python |
| 사용 툴 & 라이브러리 | PyCharm, Pico2D, Git |
| 제작 기간 | 2024.11 ~ 2024.12 |

# 감사합니다.